AGRIGUARD-FARMING AI AGENT

By

Muhammad Hamza
22-Arid-830

Abdul Basit
22-Arid-743

Muhammad Awais
22-Arid-3103

Supervisor
Mr. Imran Khurram

University Institute of Information Technology

PMAS-Arid Agriculture University Rawalpindi

Pakistan-2026

DECLARATION

We hereby declare that the Final Year Project titled “Agriguard” along with all associated documentation, designs, system architecture, and implementation details, is the result of our own independent effort, research, and development.

We further affirm that no portion of this project whether system design, source code, documentation, AI workflows, or research content has been copied, plagiarized, or reproduced from any external source without proper acknowledgment. Any material, idea, or concept taken from previously published work, research papers, tools, frameworks, or online resources has been duly referenced and credited.We also confirm that this project has not been submitted, wholly or partially, for any other degree, program, certification, or academic qualification at this or any other university, institute, or organization.

If at any point it is found that any portion of this work violates academic integrity or has been copied from another source without proper citation, we accept full responsibility and are liable for the consequences as per university rules and regulations.

Muhammad Hamza	**Abdul Basit**	**Muhammad Awais**
---	---	---

CERTIFICATE OF APPROVAL

It is to certify that the Final Year Project of BS (Computer Science) titled "Agriguard-Farming Ai Agent" was developed by "Muhammad Hamza 22-ARID-830", "Abdul Basit 22-ARID-743", and "Muhammad Awais 22-ARID-3103" under the supervision of Mr. Imran Khurram. In the opinion of the supervisor, this project is adequate in scope, quality, technical implementation, and academic contribution to fulfill the requirements for the degree of Bachelor of Science in Computer Science (BSCS).

Mr. Imran Khurram
Supervisor

Dr. Asif Nawaz
Examiner-I

Miss Sadia Zar
Examiner-II

Prof. Dr. Yaser Hafeez
Director UIIT

Executive Summary

AgriGuard is an innovative Final Year Project developed as an AI-driven smart agriculture platform designed to assist farmers and inexperienced growers in identifying plants, monitoring crop health, and making informed agricultural decisions. By leveraging deep learning, computer vision, and intelligent analysis techniques, AgriGuard enables users to identify plant species, assess growth stages, detect pests and diseases, and receive appropriate treatment recommendations through a mobile-based solution. The system aims to enhance agricultural productivity, reduce crop losses, and promote sustainable farming practices by providing accurate and timely insights.In today's agricultural environment, farmers face challenges such as limited access to expert guidance, delayed disease detection, improper pesticide usage, and reliance on traditional manual inspection methods. These issues often result in reduced crop yield and financial losses. Existing agricultural advisory systems lack automation, personalization, and real-time intelligence. AgriGuard addresses these challenges by integrating AI-powered image recognition, allowing users to capture or upload plant images and receive instant diagnostic feedback without the need for constant expert consultation.The system utilizes pre-trained deep learning models such as MobileNetV2, fine-tuned on plant image datasets, to perform accurate plant identification and health analysis. Multiple AI agents work collaboratively to analyze different aspects of plant health, including species identification, growth-stage evaluation, pest detection, and disease diagnosis. Based on these analyses, AgriGuard generates actionable recommendations such as preventive measures, pesticide selection, and usage guidelines. A comprehensive report is automatically generated to summarize findings and suggested actions for the user.

Acknowledgement

All praise is for Almighty Allah, whose countless blessings, guidance, and wisdom enabled us to complete this Final Year Project successfully. With His grace, we were able to learn, work, and overcome the challenges that arose throughout the development of this system.

We express our deepest gratitude to our respected supervisor, Mr. Imran Khurram, for his continuous support, expert guidance, and valuable supervision. Her constructive feedback, motivation, and encouragement played a significant role in shaping our project and helping us stay focused throughout the development process.

We are sincerely thankful to our parents and families, whose prayers, support, and unwavering belief in us have always been a source of strength. Their constant encouragement, patience, and values of dedication and hard work enabled us to persevere through every stage of this journey.

Finally, we are grateful to all individuals who directly or indirectly supported us during the development of our Agriguard-Farming Ai Agent .Their help contributed meaningfully to the successful completion of this project.

---------- ---------- ----------

Muhammad Hamza Abdul Basit Muhammad Awais

Abbreviations

SRS	Software Requirement Specification
PC	Personal Computer
API	Application Program interface
UI	User Interface
UX	User Experience
IP	Internet Protocol
SSD	System Sequence Diagram
PM	Project Manager
UML	Unified Modelling Language
HTTP	Hypertext Transfer protocol

Table Of Contents

List Of Figures

List Of Tables

Chapter 1: Introduction

Agriculture plays a vital role in economic growth, yet many farmers lack access to expert guidance for identifying plant species, growth issues, pests, and diseases. To address this challenge, AgriGuard is developed as an intelligent mobile-based assistant that uses image recognition and artificial intelligence to analyze plant health. By simply taking a photo of a plant, users receive instant identification of the plant type, along with information about its uses, benefits, and potential drawbacks. This system reduces reliance on specialists and provides fast, affordable, and convenient access to agricultural support.AgriGuard goes beyond basic plant detection by integrating multiple AI-based agents that evaluate different aspects of plant health. These agents determine growth stage, identify whether growth has stopped, detect the presence of pests, and diagnose diseases when pests are not present. Based on the analysis, the system provides actionable recommendations such as treatment methods, spray suggestions, and preventive measures. The final report summarizes the complete health status of the plant, enabling users to make informed decisions. Through this approach, AgriGuard empowers farmers and inexperienced users to protect crops, reduce losses, and improve productivity.

1.1 Brief

AgriGuard is an AI-powered Android application designed to assist farmers and individuals with little agricultural knowledge in diagnosing plant health using image recognition.The system allows users to capture or upload an image of a plant, which is then analyzed using trained machine learning models.

1.1.1 Plant Identification:

The first AI agent identifies the plant species and provides useful information such as its name, uses, advantages, and disadvantages.

1.1.2 Growth Stage Analysis:

The second agent determines whether the plant is in a healthy growth stage and provides suggestions to maintain or improve growth.

1.1.3 Growth Stop Investigation:

if the plant is not growing properly, the third agent identifies possible causes and recommends corrective actions.

1.1.4 Pest Detection:

The fourth agent detects the presence of pests. If pests are found, it recommends suitable treatments to protect the plant.

1.1.5 Disease Detection:

If no pest is detected, the fifth agent analyzes the plant for diseases and suggests appropriate sprays and their usage instructions.

1.2 Relevance to Course Modules

This project is strongly linked to various courses studied throughout the degree. It applies concepts from Artificial Intelligence and Machine Learning to train image-based identification models. Principles from Software Engineering and Software Requirement Engineering guided system planning, modular design, and requirement analysis. Additionally, Mobile Application Development and Object-Oriented Programming supported the development of the Android app, ensuring smooth integration of intelligent agents and user interaction.

Artificial Intelligence and Machine Learning are used to train deep-learning models for plant identification, pest detection, and disease analysis through image processing, feature extraction, and inference. The project follows Software Engineering principles, including requirement gathering, system design, implementation, testing, and detailed documentation based on SDLC and SRS concepts. Mobile Application Development plays a major role, as AgriGuard is built on Android with proper navigation flow, user authentication, and seamless model integration. Object-Oriented Programming concepts

such as abstraction, modularity, and reusability are applied to organize the system into functional agents that handle growth analysis, plant recognition, and disease diagnosis efficiently.

1.3 Project Background

Agriculture is one of the most important sectors of the economy, yet a large number of farmers struggle with plant identification and diagnosis of plant health issues. Traditionally, farmers rely on agricultural experts or experience-based judgment to recognize plant species, detect diseases, and determine suitable treatments. However, access to trained specialists is limited, especially in rural areas, leading to delays in diagnosis and increased crop losses. Additionally, inexperienced individuals who grow plants at home face similar challenges due to limited scientific knowledge.

In recent years, advancements in artificial intelligence and mobile technology have created new opportunities to support the agricultural sector. Image recognition and machine learning techniques enable automated analysis of plant images with high accuracy. Existing research and commercial applications focus mainly on disease recognition, leaving a gap in solutions that offer comprehensive plant health assessment, including growth stage evaluation, pest detection, and treatment recommendations.

1.4 Literature Review

Recent advancements in artificial intelligence (AI) and computer vision have significantly influenced modern agricultural practices. Researchers have explored the use of image-based deep learning models to detect plant diseases, identify plant species, and monitor crop health. Convolutional Neural Networks (CNNs) such as MobileNet, ResNet, Inception, and EfficientNet have demonstrated high accuracy in recognizing patterns from leaf images, making them suitable for plant-health assessment. Studies using benchmark datasets like PlantVillage have shown disease identification accuracy exceeding 90%, confirming the effectiveness of deep learning in agriculture.

Table 1.1 Comparison With Agriguard And Previous

Title	Machine Learning Methods and Findings	Shortcomings and Pitfalls
Plant Identification	MobileNetV2 model accurately identifies plant species from leaf images	Accuracy depends on image quality and dataset diversity
Growth-Stage Detection	Image analysis estimates whether the plant is growing normally	Limited data availability may reduce performance
Pest Detection	CNN detects pest-affected areas and provides treatment suggestions	Misclassification may occur if symptoms are visually similar
Disease Diagnosis	Deep models classify diseases and recommend sprays and control steps	Requires frequent model updates as new diseases emerge
Report Generation & Feedback	The system generates structured health reports and suggests personalized actions	Automated suggestions may lack precision without expert validation

1.5 Analysis from Literature Review

AgriGuard extends beyond traditional approaches by integrating multiple analytical stages into a single system. Unlike most research that stops after recognizing a disease, our system introduces additional AI agents to identify plant species, determine growth stage, detect pests, and provide tailored recommendations. This multi-agent approach makes AgriGuard more practical for real-world farming. While reviewed literature highlights the use of lightweight CNN models for mobile deployment, AgriGuard applies MobileNetV2 + TensorFlow Lite to ensure on-device inference, enabling farmers to use the system offline. This aligns with current research directions while enhancing accessibility.Many reviewed applications focus solely on leaf-level disease classification

1.6 Methodology and Software Lifecycle for This Project

The development of AgriGuard follows a structured and modular approach combining machine learning techniques with mobile application development. The project begins with the collection and preparation of plant images used to train an AI model for

identification, pest detection, and disease diagnosis. Image preprocessing techniques such as resizing, normalization, and augmentation are applied to improve model accuracy and reduce overfitting.

Figure 1.1 SDLC Model

1.6.1 Requirement Analysis

Functional and non-functional requirements were identified, including plant identification, growth analysis, pest detection, disease classification, and report generation.

1.6.2 System & Model Design

System architecture, workflow diagrams, and the roles of multiple AI agents were defined. The mobile interface was designed to ensure easy navigation for farmers.

1.6.3 Model Development & Training

Datasets were preprocessed, and the MobileNetV2-based classification model was trained using transfer learning. Evaluation metrics were used to assess performance.

1.6.4 Implementation

The trained model was converted to TFLite and integrated into the Android application. Individual agents were implemented to handle separate analysis tasks.

1.6.5 Testing

The system underwent unit testing, model accuracy testing, and UI testing. Errors were fixed continuously throughout short development cycles.

1.6.6 Deployment

The application was deployed on Android devices for field-level usage. The system operates offline with on-device inference.

1.6.7 Maintenance & Improvements

Based on feedback, new datasets, improved models, and updated recommendations can be added easily. The modular structure supports future enhancements.

1.6.8 Rationale behind Selected Methodology

The methodology used in AgriGuard combines deep learning–based analysis with modular Android development to effectively address the problem of plant identification .

Chapter 2: Problem Definition

2.1 Purpose

Many farmers and inexperienced plant growers struggle to identify plant species, understand whether their plants are growing properly, and recognize symptoms of pests or diseases. Limited access to agricultural experts, lack of technical knowledge, and delayed detection often result in ineffective decision-making and poor crop management. These challenges highlight the need for a reliable and accessible tool that can guide users in monitoring plant health quickly and accurately. AgriGuard aims to deliver an AI-based mobile application that analyzes plant images to identify the plant type, determine its growth stage, detect pests, diagnose diseases, and recommend appropriate treatments. By automating these tasks, the system empowers users with actionable insights, helping them take timely corrective measures, prevent crop losses, and improve overall productivity.

2.2 Product Functions

The AgriGuard system functions as an intelligent mobile application that helps users analyze the health and characteristics of plants through image recognition. Users upload or capture a plant leaf image, and the system identifies the plant species, evaluates its growth stage, and detects the presence of pests or diseases. Based on the analysis, the app provides suggestions, treatment recommendations, and preventive measures.The system also offers user authentication, maintains user history, and generates a detailed report summarizing the plant condition and recommended actions. By combining AI-based diagnosis with mobile accessibility, AgriGuard assists farmers and plant owners in making informed decisions to improve plant health and reduce crop loss.

2.3 Proposed Architecture

The proposed architecture of AgriGuard begins with the user capturing or uploading a plant leaf image through the mobile application, after which the image is preprocessed and analyzed by a pretrained deep-learning model (MobileNetV2) to identify the plant

species, detect pests, and diagnose diseases. The results are interpreted by specialized agents responsible for plant identification, growth-stage evaluation, pest detection, and disease diagnosis, which together determine the plant's condition. Based on the analysis, a recommendation module provides treatment suggestions, preventive measures, and care instructions, while a report generator compiles the results for user review. The system supports offline processing through TensorFlow Lite and follows a modular design to ensure scalability, efficient performance, and seamless integration between the mobile application and backend AI components.

2.4 Project Deliverables

The primary deliverable of the AgriGuard project is a fully functional Android-based mobile application capable of identifying plant species, detecting growth stages, recognizing pest attacks, diagnosing diseases, and providing suitable recommendations. A trained deep-learning model based on MobileNetV2, along with its optimized TensorFlow Lite version for mobile deployment, is also delivered as part of the system. The project further provides a multi-agent analysis framework that handles plant identification, growth evaluation, pest detection, and disease diagnosis, enabling a comprehensive assessment of plant health. Additional deliverables include a user authentication module for secure access, automated report generation summarizing analysis results, complete development documentation such as the project report, SRS, and user manual, as well as the full source code for both the Android application and the machine-learning model training process. These deliverables collectively ensure that the AgriGuard solution is functional, maintainable, and capable of supporting future enhancements.

2.5 Project development requirement

The development of the AgriGuard system requires Android Studio for building the mobile application, along with Python and TensorFlow/TensorFlow Lite for training and deploying deep-learning models. Supporting libraries such as NumPy, Keras, OpenCV, and Matplotlib are needed for image preprocessing and model evaluation, while

platforms like Google Colab or Kaggle provide GPU-based training environments. A suitable plant image dataset containing various species, healthy leaves, pest-affected, and disease-infected samples is required for model training. Hardware resources include a laptop capable of running Android Studio and training models, preferably with GPU support, and an Android smartphone for testing. Functionally, the system must allow image upload or capture, identify plant types, detect growth stages, pests, and diseases, and generate reports. Non-functional requirements include a user-friendly interface, fast processing, offline functionality using TFLite, reliability, and scalability for future expansion.

2.6 Assumptions and Dependencies

The AgriGuard system assumes that users will provide clear and focused images of plant leaves for accurate identification and diagnosis. It is also assumed that the dataset used to train the model is representative of real-world plant species, pests, and diseases. The system depends on the availability of a well-structured dataset, proper labeling, and reliable training tools such as TensorFlow and Python. Although the app supports offline inference through TensorFlow Lite, the initial model training depends on cloud platforms like Google Colab or Kaggle with GPU support. The solution also relies on Android OS compatibility, device camera functionality for capturing images, and minimal device storage to run the model efficiently. Additionally, future updates may depend on access to agricultural domain knowledge and new datasets to maintain accuracy as new diseases or pests emerge.

Table 2.1 Hardware Platform

Component	Specification	Purpose
Processor (CPU)	intel Core i5 (8th Gen or equivalent, 4+ cores)	Handles backend agent orchestration (Tflite), and frontend compilation (xml,java) during development
Memory (RAM)	8GB (DDR4, expandable to 16GB)	Supports multitasking, including running MongoDB locally, Python for testing, and browser based UI

		previews.
Storage	256GB SSD (NVMe preferred)	Stores project code, node_modules, virtual environments, and temporary scraped datasets; SSD ensures fast I/O for database queries.
Network	Gigabit Ethernet/Wi-Fi 6, stable 50Mbps+ internet	Essential for API calls, cloud syncing (GitHub/AWS), and real-time report

Table 2.2 Operating System And Version

Component	Specfication	Purpose
Operating System (Development)	Windows 10/11 (64-bit),	Provides a unified environment for coding in VS Code, running local servers (Flask), and testing UI responsiveness; supports WSL2 on Windows for Linux-like parity.
Python Version	3.12.3	Powers agentic AI (tflite), and API backend (Flask); ensures access to latest async features for real-time price fetching.
Node.js version	18.20	Powers agentic AI (tflite), web scraping

Chapter 3: Requirement Analysis

3.1 Functional Requirements

A functional requirement is a specific feature or function that a system must perform to meet user needs. It describes what the system should do, such as accepting input, processing data, and producing output.The system must allow users to register and log in securely.

3.1.1 Image Capture and Upload

The system shall allow users to capture plant images using the device camera or upload existing images from the device gallery. The uploaded images should be of sufficient quality to ensure accurate analysis and disease detection.

3.1.2 Plant Species Identification

The system shall automatically identify the plant species from the uploaded image using AI-based image recognition techniques. The identified plant name shall be displayed to the user before further analysis begins.

3.1.3 Plant Growth Stage Analysis

The system shall analyze the plant's growth stage and determine whether its growth is normal, delayed, or unhealthy. Based on the analysis, the application shall provide appropriate growth-related feedback.

3.1.4 Pest Detection

The system shall detect the presence of pests on the plant using advanced image analysis. If pests are identified, the application shall classify the pest type and estimate the severity of infestation.

3.1.5 Plant Disease Detection

If no pests are detected, the system shall analyze the plant image for diseases using a trained AI model. The application shall identify the disease name, confidence level, and affected plant area.

3.1.6 Treatment and Prevention Recommendations

The system shall provide suitable treatment recommendations based on the detected disease or pest. Recommendations may include pesticides, fertilizers, preventive measures, and farming best practices to improve plant health.

3.1.7 Plant Information Display

The system shall display comprehensive information about the identified plant, including its scientific and common name, uses, benefits, cultivation practices, and possible limitations or disadvantages.

3.1.8 Growth Improvement Suggestions

If abnormal or slow plant growth is detected, the system shall provide personalized suggestions to improve plant health, including irrigation advice, nutrient management, soil improvement techniques, and environmental recommendations.

3.1.9 Analysis Report Generation

The system shall generate a detailed analysis report summarizing the plant species, detected disease or pest, growth condition, treatment recommendations, and overall plant health. The report should be easy to understand and available for future reference.

3.1.10 Prediction History Management

The system shall maintain a history of previous plant analyses, allowing users to review past predictions, reports, and recommendations whenever required.

3.1.11 Secure Data Storage

The system shall securely store user information, uploaded images, analysis results, and activity records in the database while ensuring data privacy and integrity.

3.1.12 Administrator and Expert Access

The system shall provide authorized administrators or agricultural experts with access to user analysis reports when necessary for monitoring, research, or providing additional guidance, while maintaining appropriate access control and security.

3.1.13 Notifications and Alerts

The system shall send timely notifications and alerts regarding disease detection results, treatment reminders, weather-based warnings, application updates, and other important farming recommendations.

3.1.14 AI-Based Multi-Agent Analysis

The system shall support a multi-agent AI architecture in which specialized AI agents collaborate to perform plant species identification, pest detection, disease diagnosis, growth analysis, recommendation generation, and report preparation. This collaborative approach shall improve the accuracy, reliability, and efficiency of the overall plant health assessment.

This format is suitable for a final-year project SRS and looks much more professional than simple bullet points.

3.2 Non-Functional Requirements

Named as quality attributes of a system, they form user experience and imply some global, abstract expectations from the system. Non-functional requirements may derive from a sum of functional requirements and are implemented as a sum of app features.

3.2.1 Usability

The application must provide an intuitive, easy-to-use interface so that farmers and non-technical users can operate it without difficulty. Icons, instructions, and navigation should be clear and user-friendly.

3.2.2 Performance

The system must respond quickly to user actions, especially when processing uploaded images. Predictions should be generated within a reasonable time to ensure efficient functionality.

3.2.3 Reliability

The system must consistently provide accurate plant identification and analysis results. It should maintain reliable performance even under heavy usage or when processing multiple images.

3.2.4 Scalability

The system must be able to handle increased numbers of users, plants, and datasets without performance degradation. As the application grows, the architecture should support expanding data and model improvements.

3.2.5 Security

User data stored for login, history, and analysis must be protected through appropriate authentication and encryption measures. Only authorized users should have access to their personal information.

3.2.6 Maintainability

The application code should be modular and structured so that updates, bug fixes, and improvements can be implemented easily without affecting other components.

3.3 Use Cases

In the Unified Modeling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system. Following are the use cases of the Agriguard.

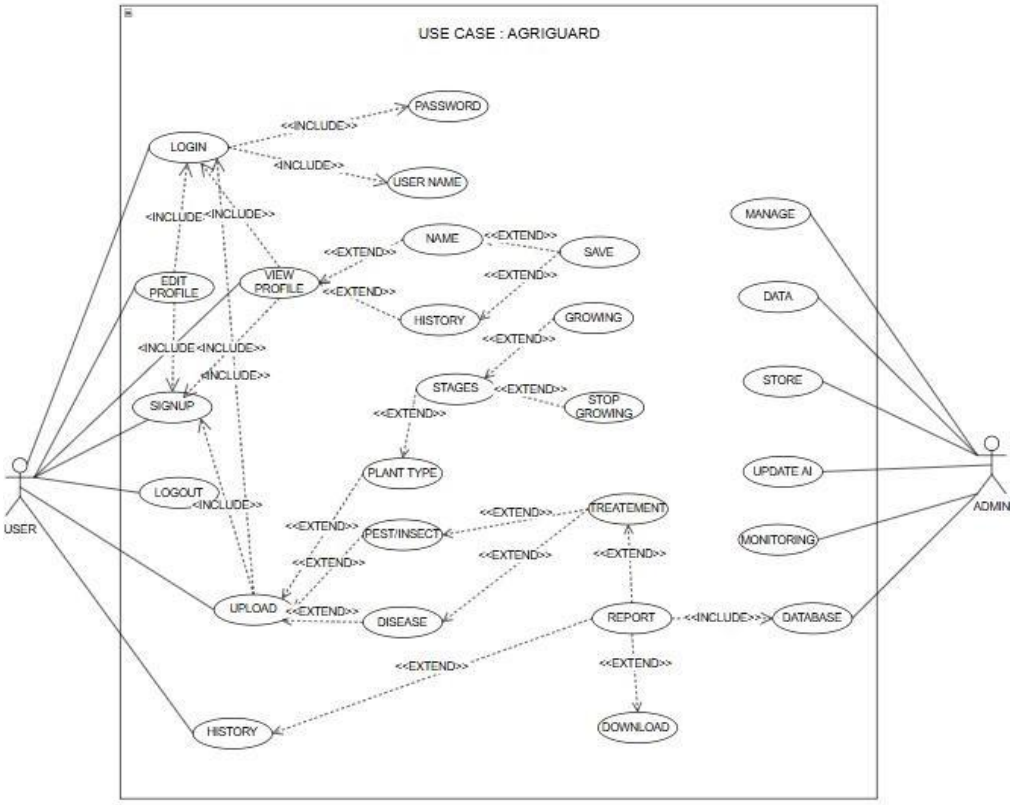

Figure 3.1 Use Case Diagram

Table 3.1 Upload Plant Image

Use Case ID:	UC-3.1
Use Case Name:	Upload plant image
Actors:	Primary Actor: Instructor Secondary Actor: System Administrator
Description:	The user uploads a plant image (leaf/fruit/whole plant) to identify

	plant name, detect diseases, pests, and growth status. The system validates, processes, and stores the image for further analysis.
Trigger:	User selects **"**Upload Plant Image" from the app interface.
Preconditions:	• User must be logged in. • The uploaded file must be a valid image format (jpg, jpeg, png).
Postconditions:	• Image is successfully uploaded and stored. • Image sent for processing by AI model. • Confirmation message displayed to the user.
Normal Flow:	• User logs into the system. • User selects Upload Plant Image option. • User selects image from gallery or camera. • System validates image type and size. • System uploads image to server/local processing. • System confirms successful upload. • Use case ends.
Alternative Flows: Invalid image format	Alternative Flows 1 : [Invalid image format] If the uploaded image is not in supported format: • System displays error: "Invalid image format." • User re-uploads a valid image. • Resume at Step 4. Alternative Flow 2 – Blurry/Low-quality Image If system finds blurred or unclear image: • System displays alert: "Image unclear. Please upload a clearer picture." • User re-uploads image.

	• Resume at Step 4.
Exceptions:	Exception 1 – File Size Too Large If image size exceeds allowed limit: • System shows: "Image size exceeds limit (max 10MB)." • User compresses image and re-uploads. Exception 2 – Corrupted Image If file is unreadable or corrupted: • System shows: "Image corrupted. Try again." • User uploads another image.
Includes:	(Authenticate User)
Special Requirements:	• Max image size**:** 10MB • Upload time under 5 seconds (normal connection)
Assumptions:	• User can correctly capture and upload plant photos. • Device camera is functional.

Table 3.2 Evaluate Plant Health Using Ai

Use Case ID:	UC-3.2
Use Case Name:	Evaluate Plant Health Using AI
Actors:	• Primary Actor: System (AI Engine) • Secondary Actor: User (Farmer)
Description:	The system analyzes the uploaded plant image using an AI model to identify the plant species, growth stage, presence of pests, and possible diseases. It then generates health insights and recommendations based on the detected conditions.
Trigger:	User selects "Start Analysis" after uploading the plant image.

Preconditions:	• User must have uploaded a valid plant image. • AI model must be configured and available.
Postconditions:	• Plant health evaluation results are generated and stored. • User is notified with a detailed report.
Normal Flow:	• User selects plant image for analysis. • System processes the image using AI model. • AI detects plant species, growth stage, pests, or diseases. • System generates health results and recommendations. • System stores the results in database. • User views the summary report.
Alternative Flows:	AI Processing Failure If AI evaluation fails system retries three times if still unsuccessful, system notifies user to try again later.
Exceptions:	• Poor-quality image flagged for re-capture • Unknown plant species marked for manual review
Includes:	UC-3.1 (Upload Plant Image)
Special Requirements:	Processing time under 10 seconds per image
Assumptions:	• Stable device performance for local AI inference • Image is clear enough for accurate analysis

Table 3.3 Generate And Display Plant Health Report

Use Case ID:	UC-3.3
Use Case Name:	Generate and Display Plant Health Report
Actors:	Primary Actor: User (Farmer) Secondary Actor: System
Description:	After AI analysis is completed, the system compiles the detected plant species, growth stage, pest or disease information, and recommended treatments into a structured report. The results are displayed to the user in graphical and tabular formats for easy

	understanding.
Trigger:	User clicks “Generate Report” after analysis is completed.
Preconditions:	• AI plant analysis must be completed. • Plant analysis data must exist in the system database.
Postconditions:	• Final report is generated, displayed, and stored. • User can view complete analysis details and recommendations.
Normal Flow:	• User initiates report generation. • System compiles plant identity, growth stage, pest/disease findings, and suggestions. • System formats report with text, charts, and summary. • Report is displayed to user. • User may export or save report.
Alternative Flows: [Alternative Flow 1 – Not in Network]	Incomplete Analysis Data If system finds missing or inconsistent analysis results System requests re-analysis or alerts user.
Exceptions:	Report generation timeout System retries once; if unsuccessful, logs the issue and notifies user.
Includes:	UC-3.2 (Evaluate Plant Health Using AI)
Special Requirements:	Report must be generated within 5 seconds
Assumptions:	• User has stable device functionality. • Analysis data is complete and available.
Notes and Issues:	Future version may automatically send report to user email or cloud storage.

Chapter 4: Design and Architecture

4.1 Design Overview

AgriGuard's design focuses on simplicity, reliability, and scalability to support smart agricultural monitoring and decision-making. The system follows a layered architecture that separates the user interface, application logic, data processing, and storage layers, making the system easier to maintain and extend. A mobile-centric design ensures that farmers can interact with the system easily through an intuitive interface, while backend services securely handle data collection, analysis, and processing. Intelligent algorithms analyze crop-related data to generate real-time insights, which are then stored efficiently using structured databases for future reference. The overall design emphasizes accuracy, data security, and ease of use to support sustainable and efficient farming practices.

4.1.1 Frontend Design

The AgriGuard frontend is developed as a mobile-first application to provide farmers with an easy, reliable, and accessible user experience. The interface is designed with a clean and simple UI, focusing on clarity and ease of navigation, so users with varying levels of technical knowledge can use it comfortably. Key screens include a Home Dashboard that displays crop status, alerts, and recommendations; a Crop Monitoring Screen showing visual indicators of plant health and growth conditions; a Weather and Alerts Screen for real-time notifications; and a Profile/Settings Screen where users can manage preferences and language options. Navigation is kept straightforward using bottom tabs for quick access to main features, and the design supports light themes for outdoor visibility. Overall, the frontend prioritizes usability, readability, and quick access to critical agricultural information.

4.1.2 Backend and API Design

The AgriGuard backend is designed using a Restful API architecture to ensure secure, scalable, and efficient communication between the mobile application and server. The backend handles data collection, processing, and storage related to crop monitoring,

environmental conditions, and user interactions. Api's manage core functions such as user authentication, crop data submission, alert generation, and recommendation delivery. The system processes incoming data from sensors, user inputs, or external weather services and applies intelligent algorithms to analyze crop health and risks. Data is stored in structured databases for reliability, and proper validation, logging, and error handling mechanisms are implemented to ensure system stability and data integrity.

4.1.3 AI and ML Design

AgriGuard's AI and ML components are designed to support intelligent crop monitoring and decision-making. The system uses machine learning models to analyze agricultural data such as crop conditions, environmental factors, and historical patterns to detect risks and predict crop health. These models help generate timely recommendations related to irrigation, pest control, and disease prevention. AI-based analysis also supports early warning alerts by identifying unusual patterns that may indicate crop stress or potential yield loss. The design allows models to be updated and improved over time, ensuring accurate insights that help farmers make better, data-driven decisions.

4.1.4 Database and Data Flow Design

It uses a structured and efficient data storage approach to manage agricultural information reliably. User data, crop records, and system configurations are stored in relational databases to ensure consistency and accuracy, while sensor readings and environmental data are handled as time-based records for trend analysis. Data flows from the mobile application and external sources, such as weather services or sensors, to the backend through secure APIs, where it is validated and processed. The processed data is then analyzed by intelligent modules to generate insights and alerts, which are stored and sent back to users in real time. Regular backups and proper data management practices ensure data safety, availability, and long-term system reliability.

4.1.5 Security and Deployment Design

AgriGuard's security and deployment design focuses on protecting user data and ensuring reliable system availability. Secure authentication and authorization mechanisms are used to control access, and data is protected through encryption during transmission and storage. Input validation and secure coding practices help defend against common security threats, ensuring the system remains safe for users. The application is deployed on a scalable cloud infrastructure that supports smooth updates and continuous operation, with regular testing and monitoring to maintain performance and stability. This design allows AgriGuard to grow in functionality such as adding advanced analytics or new monitoring features while maintaining data security, reliability, and ease of maintenance.

4.2 UML Structural Diagram:

This diagram represents the main actors and system interactions in AgriGuard, focusing on agricultural monitoring and decision support. The primary actor is the Farmer, who interacts with the system to monitor crops and receive recommendations. A secondary actor is the System Admin, responsible for managing users, data, and system configurations. Key use cases include Register/Login, View Crop Status, Monitor Environmental Conditions, Receive Alerts and Recommendations, Manage Farm Profile, and View Historical Data, while the admin handles User Management, Data Management, and System Maintenance.

4.2.1 Deployment Diagram:

The deployment diagram illustrates how the system is physically deployed and executed in a real-world environment. The mobile application is deployed on farmers' smartphones, which communicate with backend services hosted on cloud servers through secure internet connections. Backend components such as the application server, database server, and AI processing modules run on cloud infrastructure to ensure scalability and reliability. External services like weather APIs or sensor gateways are also connected to the system

to provide real-time data. This diagram highlights how AgriGuard's software components are distributed across devices and servers to support efficient, secure, and continuous agricultural monitoring.

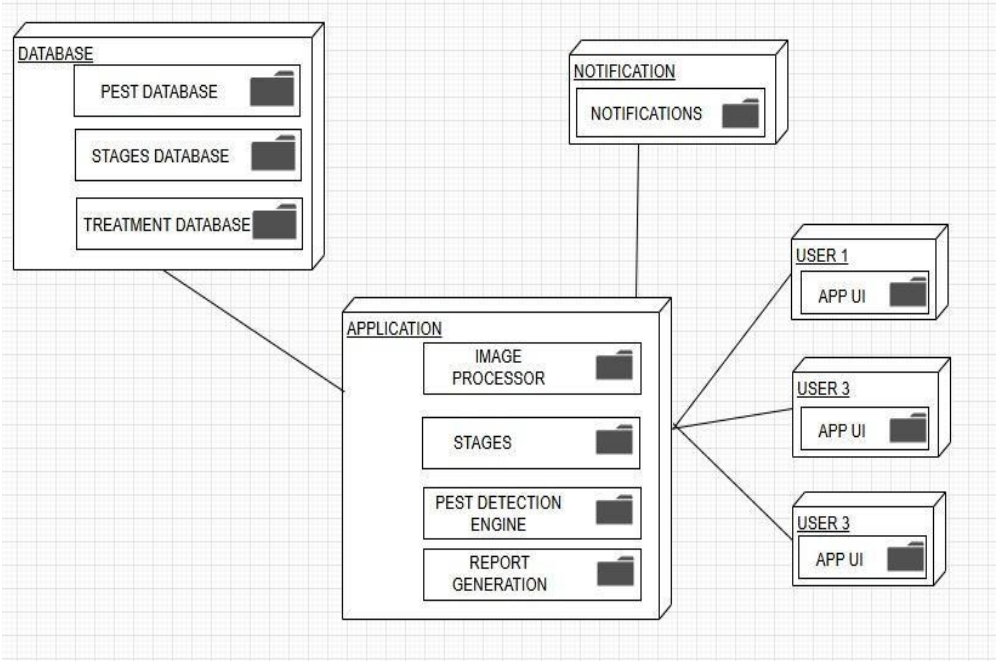

Figure 4.1 Deployment Diagram

4.2.2 Class Diagram:

This UML Class Diagram represents the main domain entities of AgriGuard and shows their attributes, operations, and relationships. The core classes include Farmer, Crop, Field, SensorData, WeatherData, and Recommendation, which together model crop monitoring and farm management activities. The AIModel class analyzes sensor and environmental data to generate insights, while Alert handles notifications for risks such as pests, diseases, or weather issues. Supporting classes like UserProfile and Admin manage system access and configurations, with associations and dependencies illustrating how data flows between farmers, crops, and intelligent decision-support components.

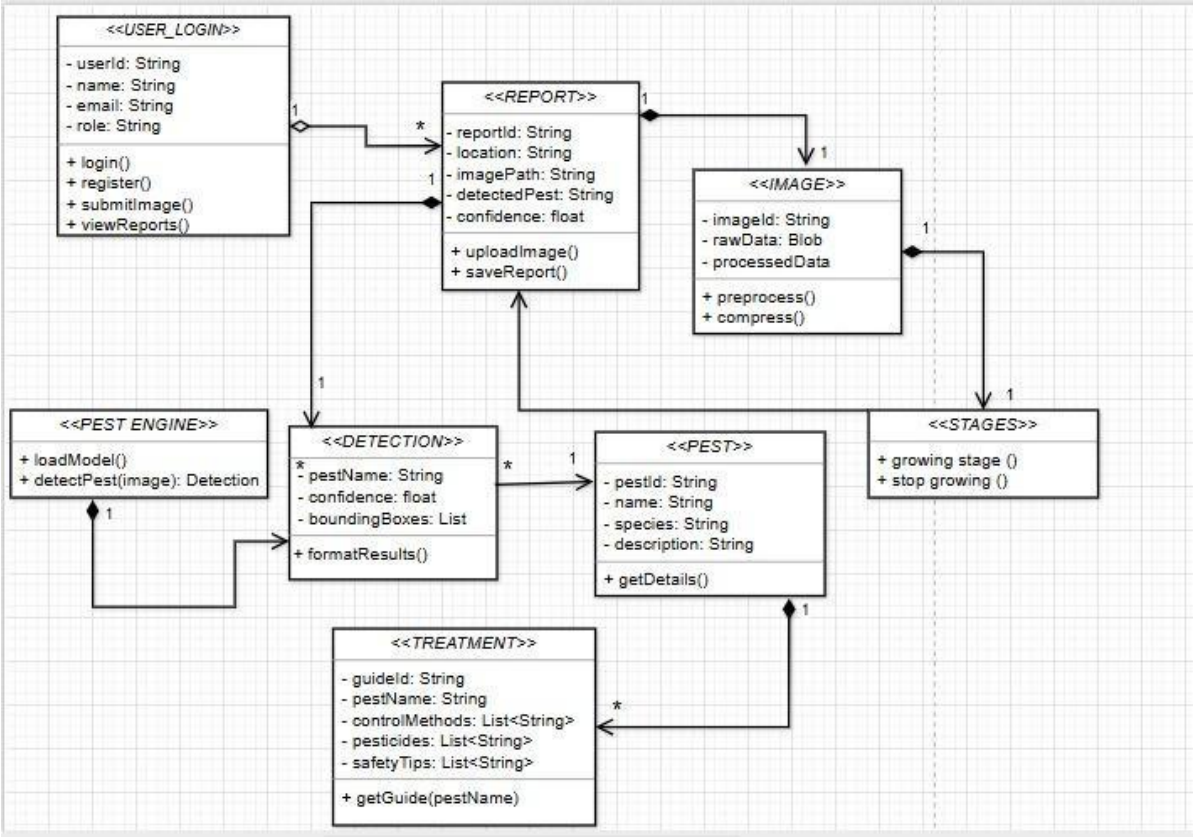

Figure 4.2 Class Diagram

4.2.3 Sequence Diagram:

A user initiating, processing, and completing a plant health analysis. It involves key participants User, Mobile Application (Android app), API Gateway, AI Analysis Service, Recommendation Service, Notification Service, and Database highlighting synchronous and asynchronous service calls, sequential AI analysis steps (plant identification, growth stage detection, pest detection, and disease diagnosis), and error handling mechanisms (such as validation responses and partial-result acknowledgments). The diagram demonstrates how user-submitted plant images are securely transmitted, analyzed by AI models, stored in the database, and returned as actionable insights and reports to the user, ensuring an efficient and reliable plant monitoring workflow.

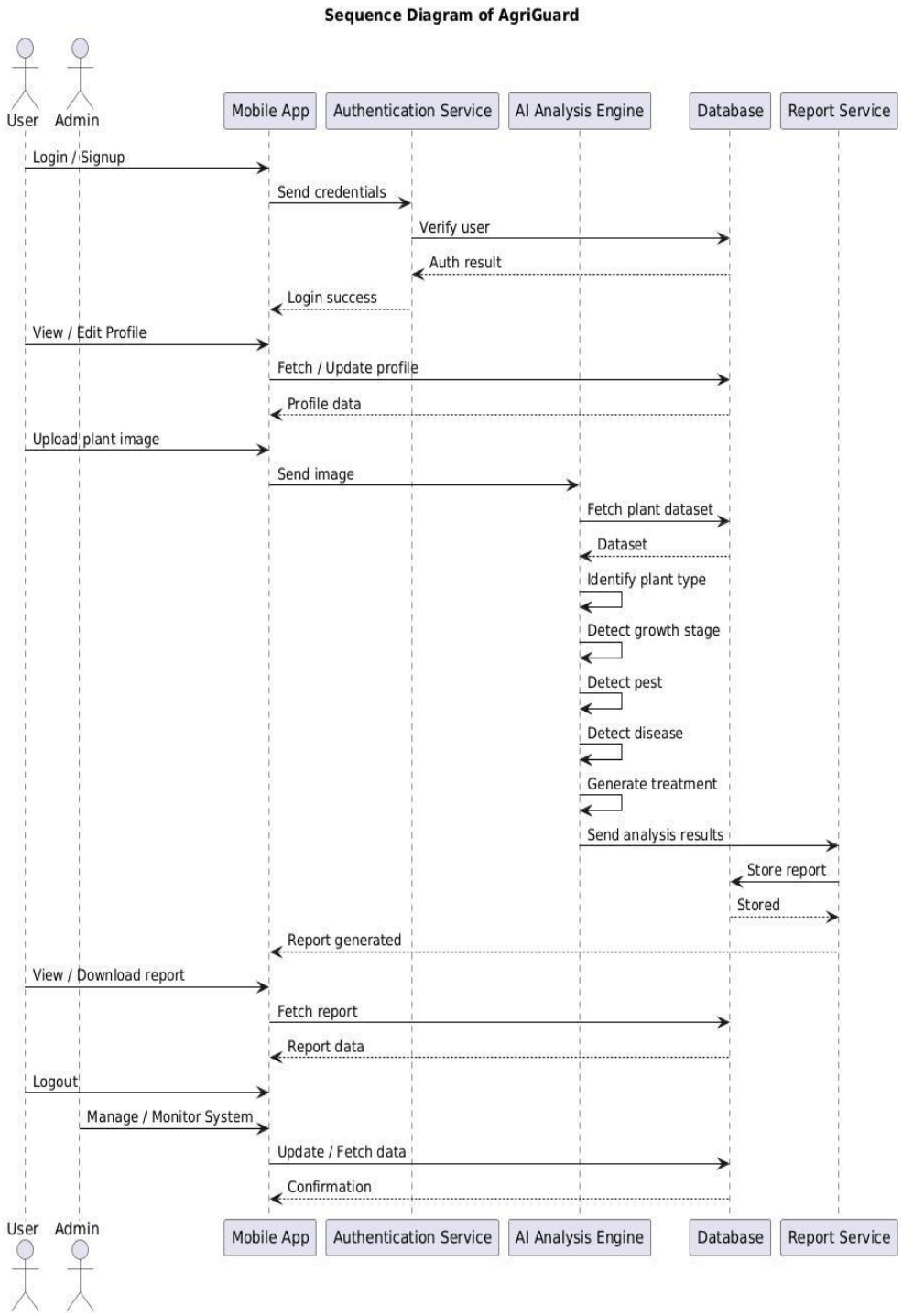

Figure 4.3 Sequence Diagram

4.2.4 Activity Diagram

The activity diagram of AgriGuard illustrates the workflow of the system from user input to disease prediction and advisory output. It shows how the farmer uploads crop images, the backend processes them using AI models, and the system generates diagnosis results. The diagram helps developers understand the step-by-step process and identify decision points for efficient system operation.

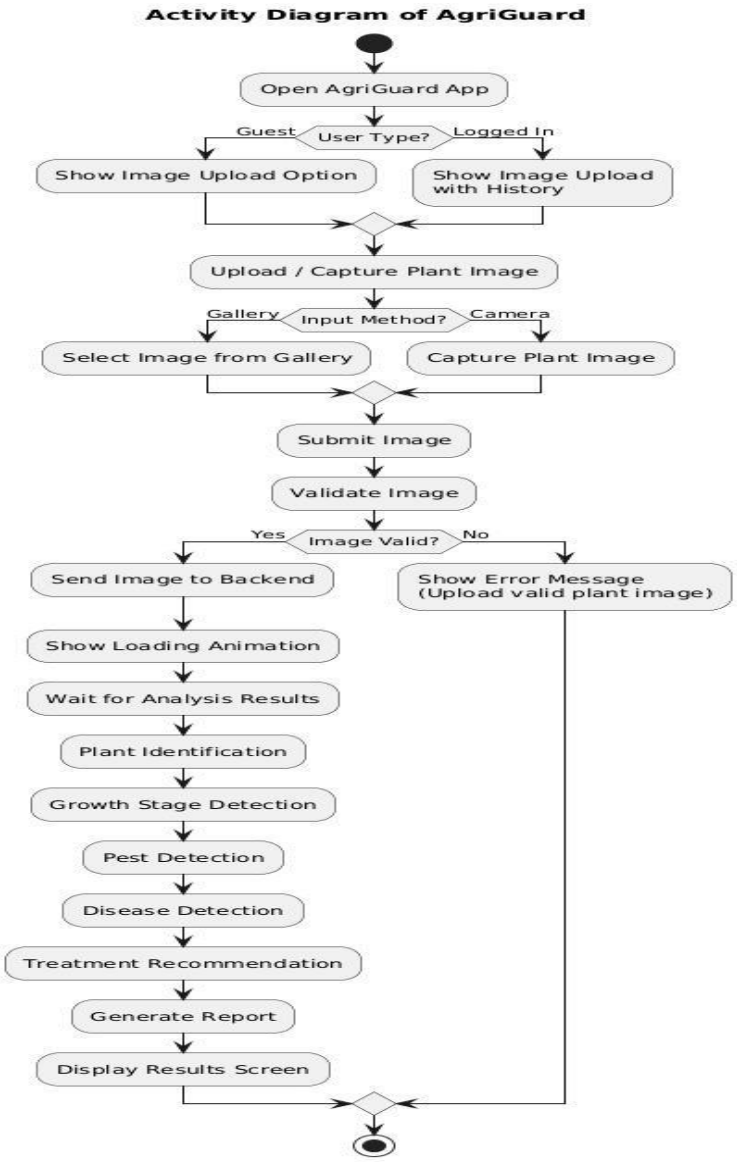

Figure 4.4 Activity Diagram

4.2.5 Collaboration Diagram

It illustrates how different system components work together to support smart agricultural monitoring and decision-making. It shows the Farmer interacting with the AgriGuard Mobile App, which communicates with the Backend Server through secure APIs. The backend collaborates with services such as Crop Monitoring, AI Analysis, Weather Service, and Alert & Notification Service to process crop and environmental data. These components exchange messages to analyze conditions, generate recommendations, store data in the database, and send alerts back to the farmer, highlighting the cooperative behavior of system objects to achieve efficient farm management.

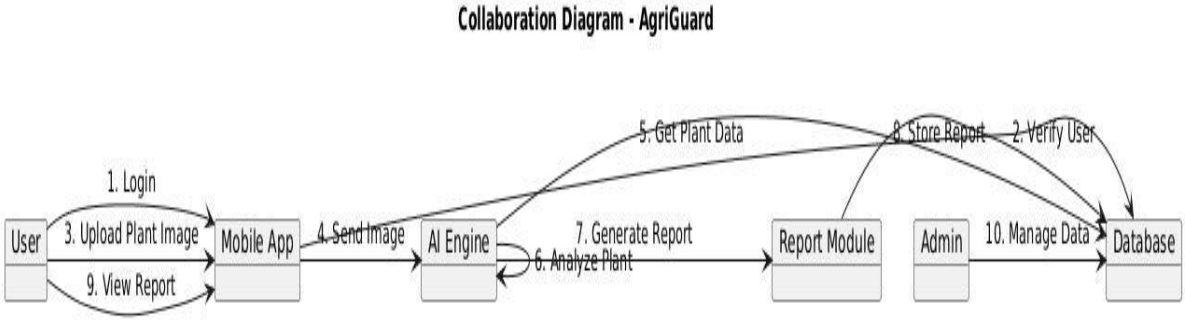

Figure 4.5 Collaboration Diagram

4.2.6 Block Diagram

It represent major subsystems as rectangular blocks and their interactions as arrows that indicate data and control flows. Unlike detailed UML diagrams (such as class or sequence diagrams), it focuses on functional decomposition and overall system flow, making it ideal for stakeholder presentations and early-stage system design. For AgriGuard, the block diagram illustrates the flow from user inputs (plant images and data) through AI-based analysis modules including plant identification, growth stage detection, pest detection, and disease diagnosis to result generation, recommendations, and notifications, emphasizing modularity, scalability, and clear separation of responsibilities

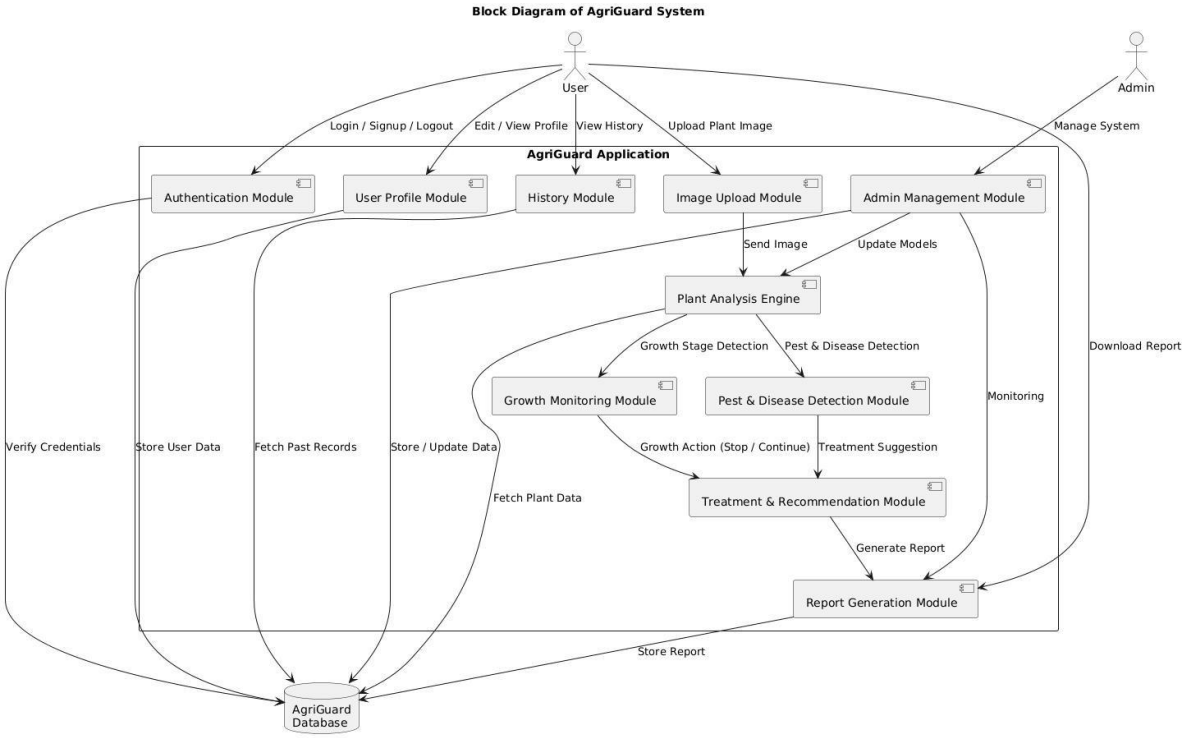

Figure 4.6 Block Diagram

4.2.7 Entity Relation Diagram

The ER Diagram for AgriGuard models the conceptual database schema by identifying key entities such as User, Plant, Image Upload, Growth Stage, Pest, Disease, Treatment, Report, and Admin, along with their attributes (with primary keys underlined) and relationships defined by cardinalities (1:1, 1:N, N:M). It captures the relational structure required to store user profiles, uploaded plant images, analysis results, detected growth stages, pests and diseases, recommended treatments, and generated reports. For example, a User (1) can upload many Plant Images (N), each Image (1) is associated with one Plant (1), and a Plant (1) may be affected by multiple Pests or Diseases (N). This database design supports normalization (e.g., 3NF), data integrity, and scalability, enabling efficient queries such as retrieving a user's analysis history, tracking plant health over time, and generating accurate treatment recommendations.

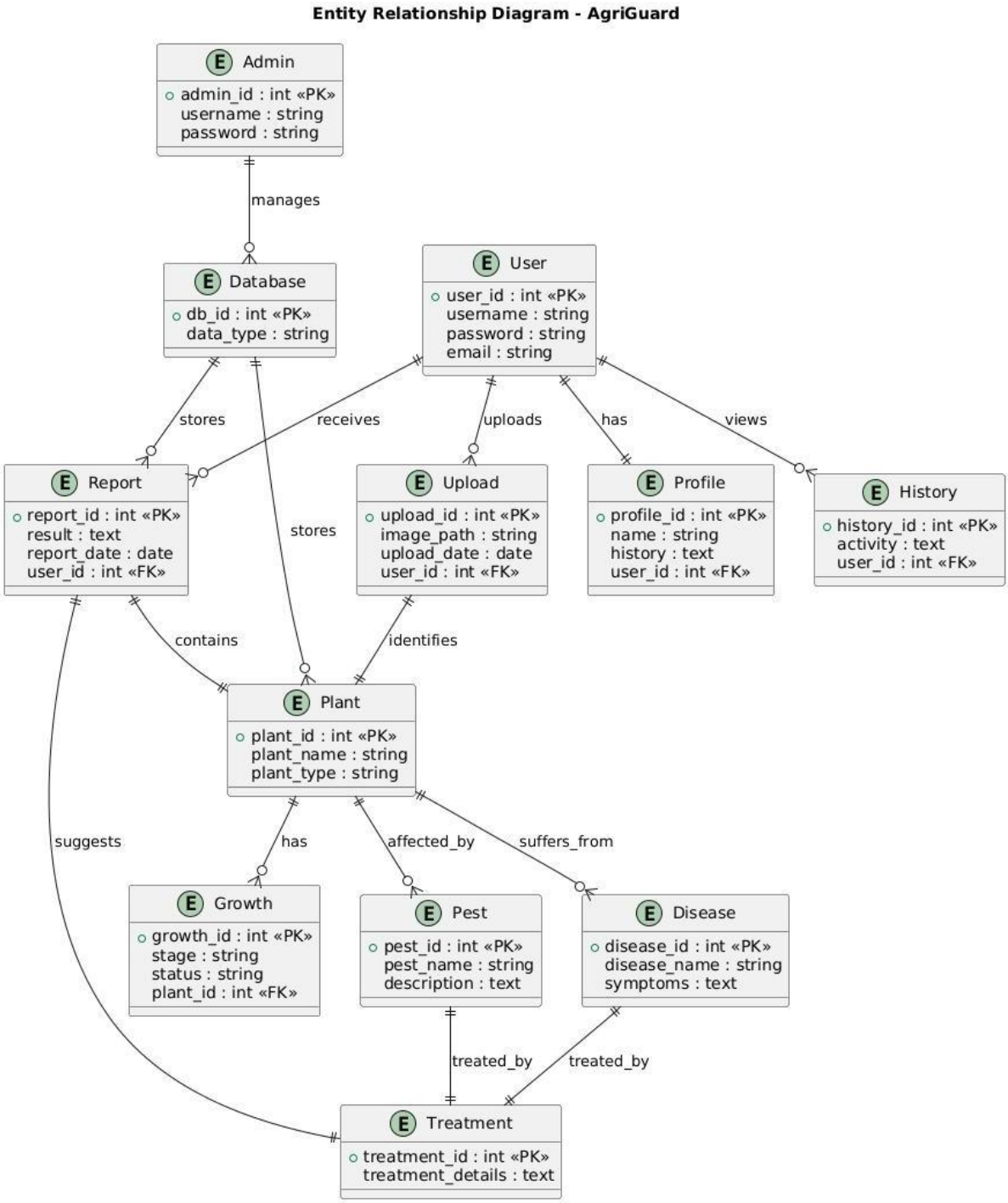

Figure 4.7 Entity Relation Diagram

4.2.8 State Transition Diagram

This UML State Machine Diagram (State Transition Diagram) models the lifecycle of a Negotiation entity in Dealify. It depicts discrete states

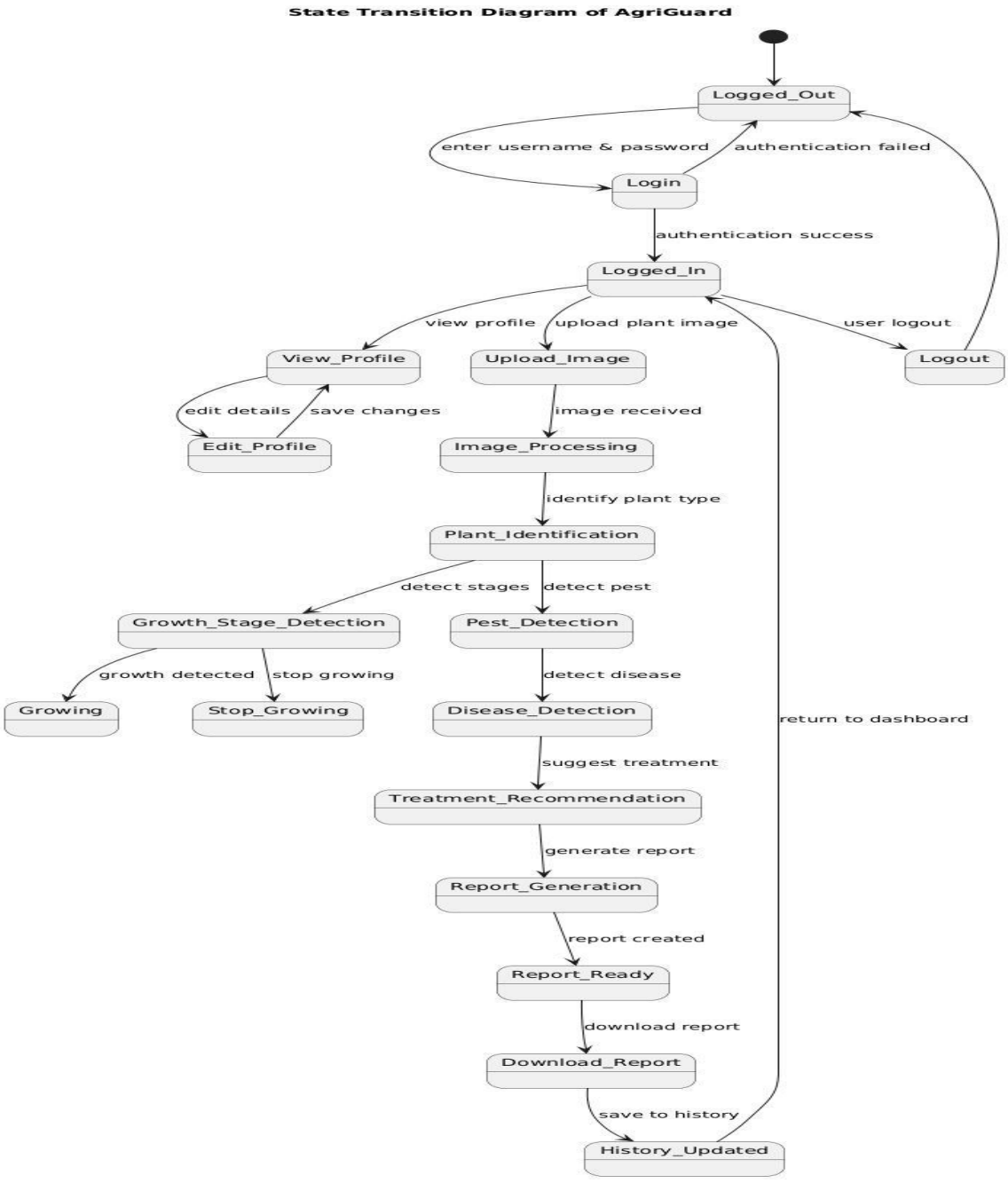

Figure 4.8 State Transition Diagram

4.2.9 Package Diagram

The Package Diagram for Dealify organizes the system into logical packages (namespaces or modules), illustrating dependencies between them. It groups related elements: e.g., Frontend depends on API & Services, which in turn depends on AI Core and Data Layer. This promotes modularity, showing how frontend UI relies on backend orchestration without direct coupling to data or AI internals. Arrows indicate import/dependency.

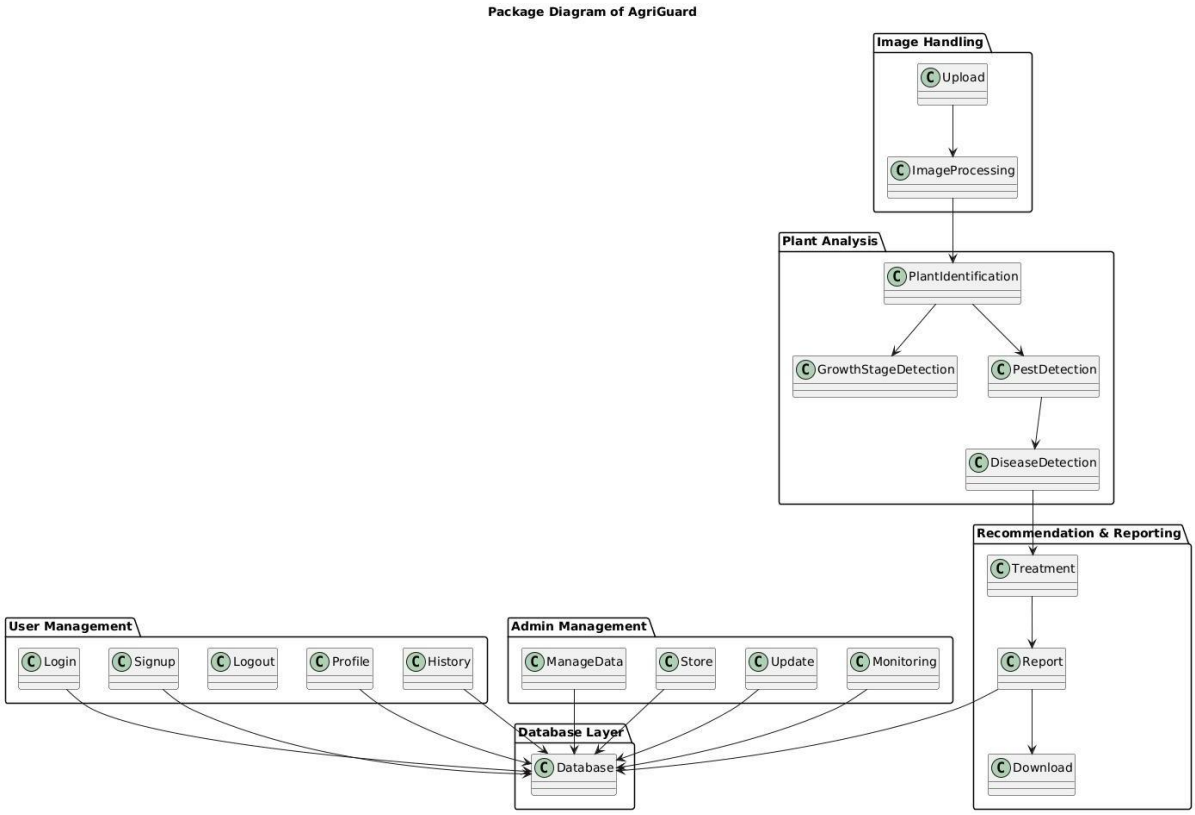

Figure 4.9 Package Diagram

Chapter 5: Implementation

5.1 Component Diagram

It illustrates how the system is organized into independent yet interconnected software components that work together to provide plant analysis services. The User interacts with the system through the Mobile App UI, while the Admin uses the Admin Panel UI for management and monitoring. Core components include Authentication for login and security, Profile Management for user data, Image Upload for submitting plant images, and the Plant Analysis module, which further handles growth stage detection, pest detection, and disease detection.

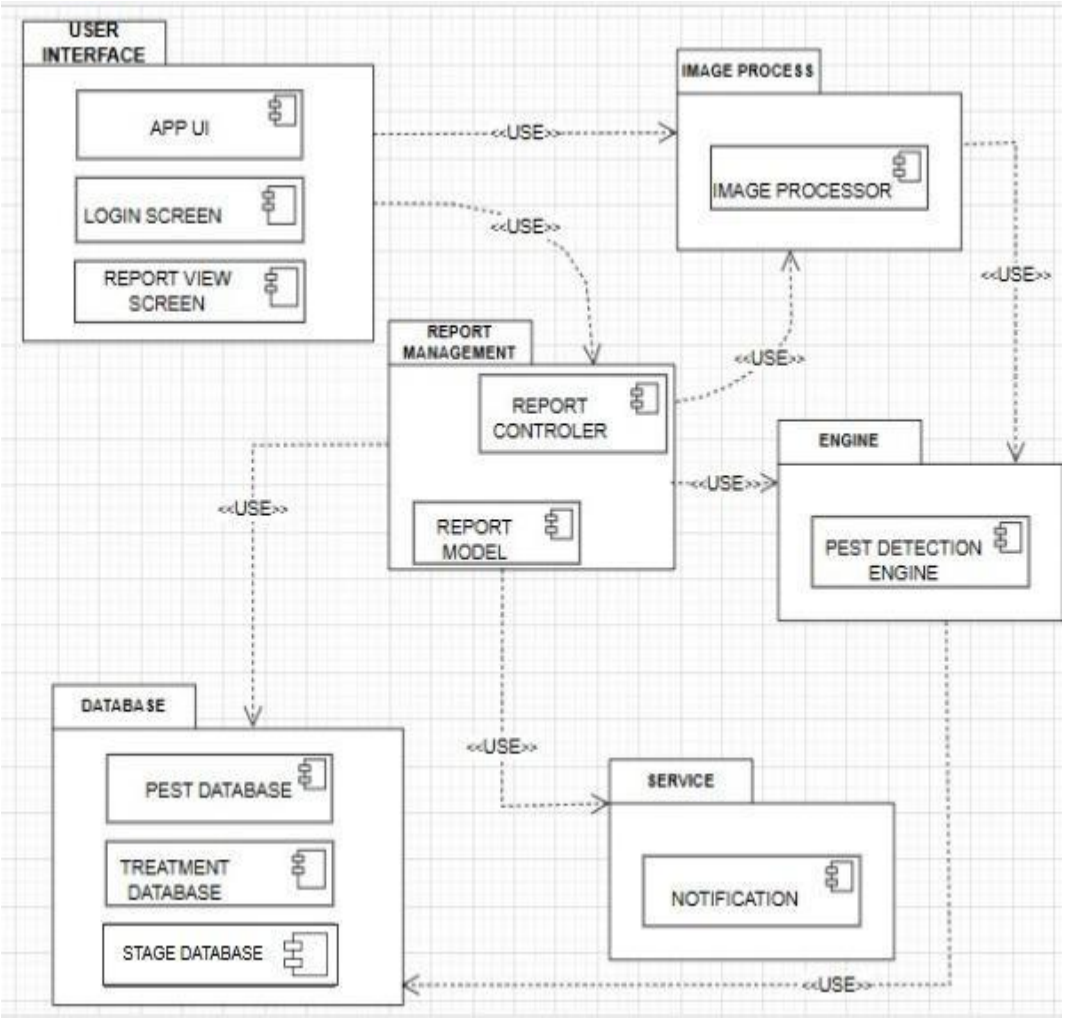

Figure 5.1 Component Diagram

5.2 Network and Protocol Choice

AgriGuard operates using a cloud-based architecture where the Android application communicates with external APIs and cloud services over the internet. The system utilizes:

- HTTPS protocol for secure communication between the mobile application and cloud services.
- RESTful APIs for efficient communication with the Groq API, Weather API, and Firebase services.
- JSON format for exchanging requests and responses between the application and external services.
- Firebase Authentication for secure user login and registration.
- Firebase Realtime Database for storing user profiles, prediction history, and application data.

These technologies ensure secure, reliable, and scalable communication between the application and cloud services.

5.3 Data Preparation

The plant disease detection module utilizes a collection of healthy and diseased plant leaf images obtained from publicly available agricultural datasets. Before analysis, images undergo preprocessing techniques such as resizing, normalization, and quality enhancement to improve prediction accuracy. The dataset includes multiple crop types and disease categories, enabling the system to identify a wide variety of plant diseases effectively. Proper preprocessing ensures consistent input quality and improves the overall performance of the AI model.

5.4 Plant Disease Detection

The disease detection module analyzes images captured through the device camera or selected from the gallery. After preprocessing, the image is processed by the AI model to identify plant diseases. The system predicts the disease name and confidence level while

also determining whether the plant appears healthy or infected. The prediction results are displayed instantly to the user along with appropriate treatment recommendations.

5.5 AI-Based Recommendation System

AgriGuard integrates the Groq API to generate intelligent farming recommendations based on disease detection results. After identifying a disease, the application sends the prediction data to the AI service, which generates structured responses including:

- Disease description
- Possible causes
- Treatment recommendations
- Preventive measures
- Farming best practices

This AI-powered recommendation system provides farmers with accurate and easy-to-understand agricultural guidance.

5.6 System Integration Pipeline

The overall workflow of AgriGuard consists of the following steps:

- The user captures or uploads a plant leaf image.
- The application preprocesses the image for analysis.
- The AI model analyzes the image and detects the plant disease.
- The prediction result and confidence level are generated.
- The result is sent to the Groq API.
- The AI generates detailed disease information and treatment recommendations.
- Weather information is retrieved using the Weather API.
- The complete analysis report is displayed to the user.
- The prediction history is securely stored in Firebase for future reference.

5.7 Weather Information Integration

AgriGuard integrates the Weather API to provide real-time weather updates relevant to farming activities. The application displays temperature, humidity, wind speed, rainfall conditions, and weather forecasts. Based on current weather conditions, farmers receive suggestions that help reduce disease risks and improve crop management. This feature enables users to make better farming decisions according to changing environmental conditions.

5.8 Deployment Prototype

The AgriGuard application was developed and deployed as an Android application using Android Studio. Firebase services were configured to manage authentication and cloud database functionality, while external APIs were integrated for weather information and AI-generated recommendations. The modular architecture allows future deployment on cloud platforms and supports easy maintenance and scalability.

5.9 Testing and Validation

The application was tested extensively on Android devices to ensure functionality, reliability, and performance. Testing included:

- User registration and login validation.
- Plant image upload and camera functionality.
- Disease detection accuracy.
- AI recommendation generation.
- Weather information retrieval.
- Firebase database operations.
- Prediction history management.
- User interface responsiveness and usability.

Functional and integration testing confirmed that all system modules operated correctly and provided accurate and consistent results.

5.10 User Interface

The AgriGuard user interface was designed to be simple, intuitive, and user-friendly, enabling farmers to access all features with minimal effort. The application allows users to:

- Register and log in securely.
- Capture plant images using the device camera.
- Upload images from the gallery.
- Detect plant diseases instantly.
- View disease details and treatment recommendations.
- Access real-time weather information.
- Read agricultural news.
- Interact with the AI-powered farming assistant.
- View previous prediction history.
- Manage personal profile information.

The application communicates with Firebase, the Groq API, and the Weather API through secure REST API requests and presents all information in a clean, responsive, and easy-to-understand interface suitable for both experienced and novice user

Figure 5.2 Splash Screen

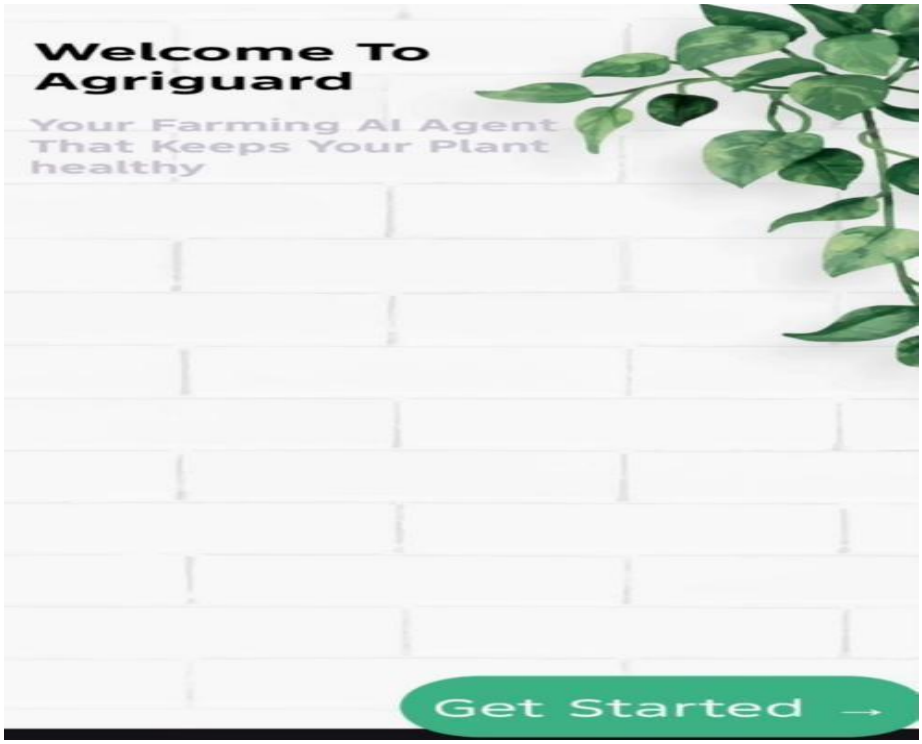

Figure 5.3 Welcome Screen

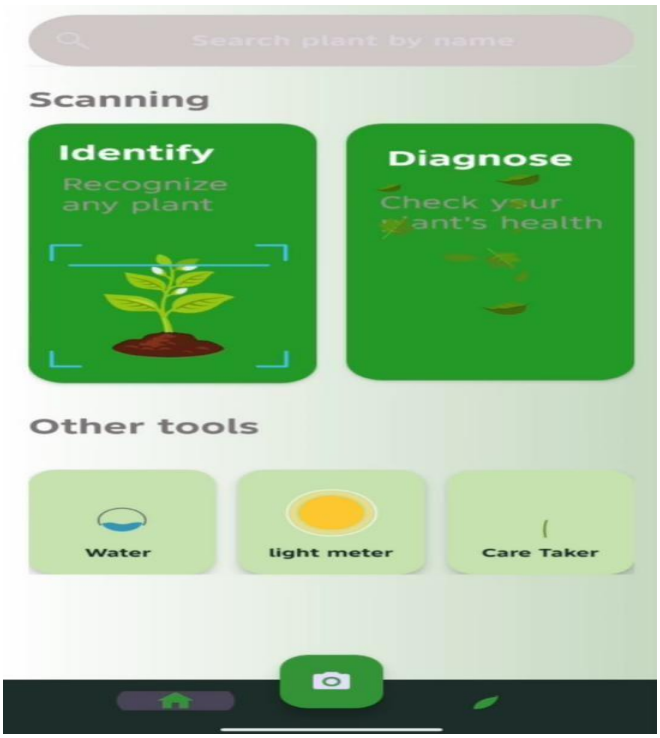

Figure 5.4 Home Screen

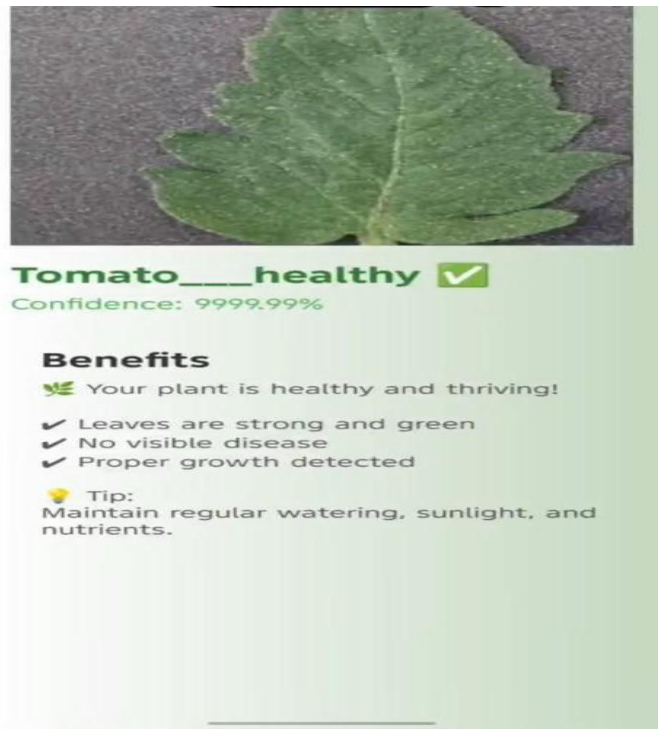

Figure 5.5 Result Screen 1

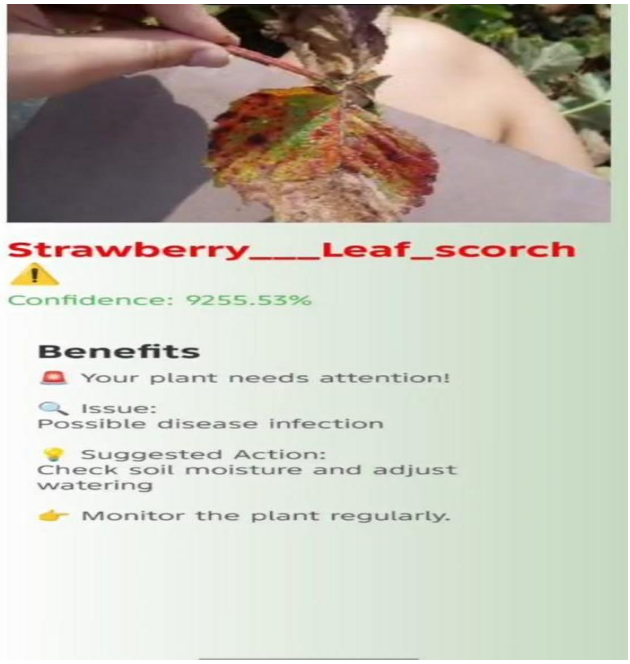

Figure 5.6 Result Screen 2

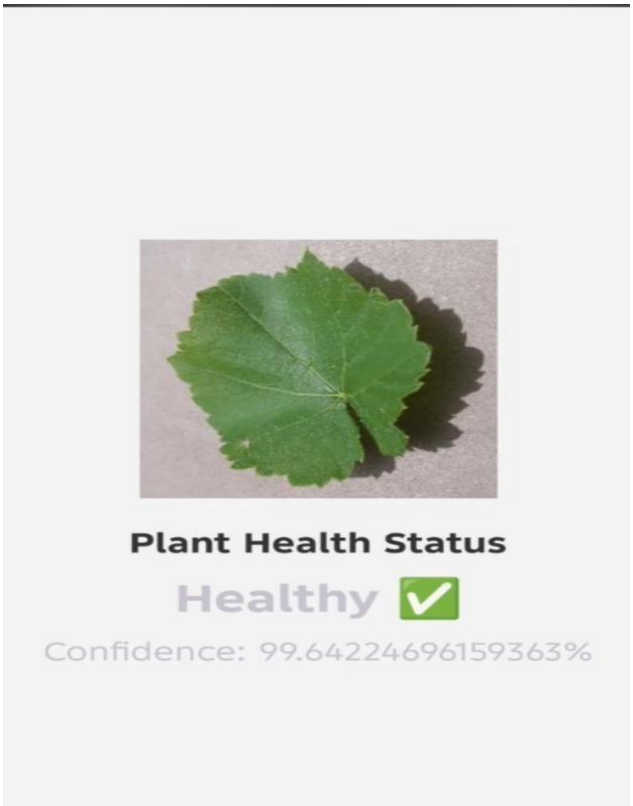

Figure 5.7 Health Screen

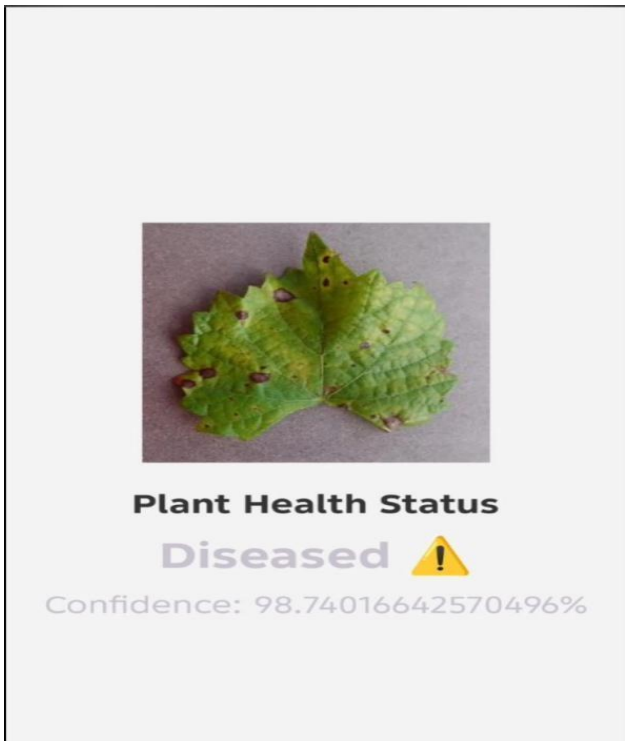

Figure 5.8 Diseased Screen

Figure 5.9 Water Test Screen

Figure 5.10 Chat Bot Screen 1

Figure 5.11 Chat Bot Screen 2

Chapter 6: Testing and Evaluation

This chapter presents the testing and evaluation process of the Agriguard. The purpose of testing is to ensure that the system meets its functional and non-functional requirements, operates reliably under different conditions, and provides accurate and usable results to end-users. Evaluation further validates the system's performance against benchmarks and user expectations.

6.1 Verification

Verification is the process of checking that all features of the AgriGuard application, such as plant disease detection, user authentication, and weather information, have been implemented according to the Software Requirement Specification (SRS) and design documents.

6.1.1 Functional Testing

Functional testing was conducted to ensure each feature of the AgriGuard system performs as expected in the context of plant disease detection and farmer support.

- Login/Register: Verified that farmers and caretakers can securely create accounts and log in using valid credentials.
- Leaf Image Upload: Confirmed that AgriGuard accepts leaf images in .jpg, .jpeg, and .png formats and successfully stores them for analysis.
- Disease Detection (Model Inference): Verified that the EfficientNet-based model correctly processes preprocessed crop images and returns the predicted disease class along with a confidence score.
- Agriguard Guidance Generation: Ensured the system generates clear, structured, and non-diagnostic crop care recommendations based on the detected disease.
- History Tracking: Confirmed that previous scans and results are properly saved and can be viewed by the user.

6.1.2 Static Testing

Static testing in AgriGuard involved systematic code reviews, XML layout inspections,

and documentation walkthroughs to identify logical errors, missing validations, and requirement mismatches before running the application. Special attention was given to API integration, model loading, and input validation to ensure reliability and maintainability.

6.2 Validation

- Validation ensures the final product meets users' needs and intended purpose. For AgriGuard, this involved evaluating the plant disease detection model's accuracy and reliability in real farming scenarios, with a target performance threshold of > 90% classification accuracy**.**
- Model Comparison AgriGuard's predictions were compared against expert-labeled agricultural datasets and verified ground-truth labels to measure correctness and consistency.

Example**:** "On the plant leaf disease dataset, AgriGuard achieved 87% accuracy**,** demonstrating strong performance for automated crop disease identification and providing reliable decision support for farmers."

6.3 Usability Testing

Usability testing for AgriGuard focused on the farmer and caretaker experience to ensure the interface is simple, clear, and easy to use for non-technical agricultural users.

- Navigation: Testers evaluated the clarity of buttons, icons, and on-screen instructions for key tasks such as capturing/uploading leaf images, viewing results, and accessing scan history.
- Responsiveness: Confirmed that the AgriGuard interface remains fully functional and properly aligned across different Android devices and screen sizes.
- User Feedback: Verified that visual indicators such as loading progress, disease confidence scores, and guidance cards help users clearly understand the analysis process and results.

6.4 Module / Unit Testing

Unit testing in AgriGuard was conducted to verify that individual components of the application function correctly in isolation before full system integration. This process ensured code reliability, simplified debugging, and improved maintainability. The authentication module was tested to confirm proper handling of valid and invalid login/register inputs. The image preprocessing module was verified to ensure uploaded leaf images are correctly resized, normalized, and converted into the required tensor format for the EfficientNet model. The model inference function was tested to confirm successful model loading and accurate return of the predicted disease class with confidence score. Additionally, the guidance generator was checked to ensure the system provides the correct structured crop-care recommendations for each detected disease, while the history storage module was tested to verify that scan results are properly saved and retrieved.

6.5 Integration Testing

Integration testing in AgriGuard was performed to verify that different modules of the system work together smoothly as a complete workflow. The testing focused on the end-to-end process from user authentication to disease detection and guidance delivery. The login/register module was integrated with the backend to ensure secure session handling. The image upload component was tested with the preprocessing pipeline and EfficientNet model to confirm that captured leaf images flow correctly through the system and produce valid predictions. The model output was then validated with the guidance generation module to ensure the correct crop-care recommendations are displayed for each detected disease. Additionally, the history feature was tested with the database/API to confirm that scan results are properly stored and retrieved. Integration testing ensured that AgriGuard operates reliably as a unified application in real-world usage.

6.6 System Testing

System testing in AgriGuard was conducted to evaluate the complete application as a fully integrated system and ensure it meets functional and performance requirements in real-world conditions. The testing covered the full user workflow, including account registration/login, leaf image capture or upload, disease detection using the EfficientNet model, guidance generation, and scan history retrieval. Performance testing confirmed that image processing and prediction occur within acceptable response times on typical Android devices. Compatibility testing ensured the application runs smoothly across different screen sizes and device configurations. Error handling and edge cases—such as invalid image formats, network interruptions, and low-confidence predictions—were also verified. Overall, system testing confirmed that AgriGuard operates reliably, efficiently, and as intended for end users.

6.7 Acceptance Testing

Acceptance testing in AgriGuard was conducted to confirm that the final system meets the requirements of its intended users, particularly farmers and agricultural caretakers. Selected end users evaluated the application in realistic scenarios, including account login, capturing or uploading crop leaf images, viewing disease detection results, and following the provided crop-care guidance. Feedback focused on ease of use, clarity of results, and overall usefulness in supporting crop health decisions. Testers confirmed that the system performs reliably and that the guidance is understandable and practical for field use. Based on user feedback and requirement verification, AgriGuard was deemed ready for deployment and real-world agricultural support.

6.8 Stress Testing

Stress testing in AgriGuard was performed to evaluate how the system behaves under extreme operating conditions and high workload scenarios. The application was tested by submitting a large number of leaf image uploads in rapid succession and by running the EfficientNet inference repeatedly to observe system stability and response time.

Additional tests simulated low-memory conditions, slow network connectivity, and prolonged continuous usage on Android devices. The results showed that AgriGuard maintained functional stability, handled peak loads without crashes, and degraded gracefully when resources were limited. Stress testing confirmed that the system is robust enough for heavy and continuous use in real-world agricultural environments.

6.9 Hardware Configuration for Testing

The hardware configuration used for testing AgriGuard was carefully selected to support the computational demands of plant disease detection and image processing. For model training and inference evaluation, high-performance cloud platforms such as Kaggle Notebooks and Google Colab were utilized, taking advantage of GPU acceleration to efficiently process the plant leaf disease dataset. On the client side, the Android application interface was tested on a range of devices, including mid-range smartphones and different screen sizes, to verify layout responsiveness and smooth user interaction. The backend and model environment were validated on systems supporting Python 3.10 along with required libraries such as TensorFlow Lite, TensorFlow, and OpenCV to ensure compatibility and stable deployment. This diverse hardware setup confirmed that AgriGuard operates reliably and efficiently across varying device capabilities and real-world usage conditions.

6.10 Evaluation

The evaluation phase for AgriGuard focused on both quantitative and qualitative performance using relevant agricultural metrics. Quantitatively, the EfficientNet-based plant disease classification model was evaluated using a confusion matrix, precision, recall, and overall accuracy to measure how effectively it handled class variations in the crop leaf dataset. Particular attention was given to performance across minority disease classes to ensure balanced detection. Qualitatively, the system was assessed for responsiveness and usability, targeting an end-to-end processing time of approximately 3–6 seconds per image on typical Android devices. The guidance generation module was also evaluated for its ability to transform raw model predictions into clear, farmer-

friendly, structured recommendations. This comprehensive evaluation demonstrated that the applied preprocessing, data balancing, and model fine-tuning strategies effectively reduced overfitting while maintaining reliable and practical crop disease decision support.

6.11 Deployment

The deployment of the AgriGuard system was designed to be modular and scalable, supporting both local testing and real-world mobile deployment. The application follows a structured client–model architecture in which the trained EfficientNet model is converted to TensorFlow Lite and embedded directly within the Android application for on-device inference. Where backend services are used, communication follows RESTful API principles to enable predictable and stateless data exchange. Secure data handling is maintained, with image data processed locally where possible and protected during any network transmission. The Android user interface was designed to provide a clean, farmer-friendly experience for capturing leaf images, viewing predicted disease labels, confidence scores, and crop-care guidance. The final deployment package includes the optimized TFLite model, image preprocessing pipeline, Android application build, and required environment configurations, ensuring AgriGuard can be reliably deployed in real agricultural scenarios.

6.12 Maintenance

Maintenance procedures for AgriGuard are established to ensure the system remains accurate, reliable, and secure as new agricultural data and evolving threats emerge. Regular updates include retraining and fine-tuning the EfficientNet-based plant disease model with expanded and more diverse crop datasets to improve prediction accuracy and reduce classification variance. The software architecture is intentionally modular, allowing developers to update individual components such as the TFLite model, guidance knowledge base, or local database without disrupting the entire application. Security maintenance involves periodic reviews of data handling practices, API security (where applicable), and user authentication mechanisms to protect user information. Performance monitoring and bug fixes are scheduled through version updates to maintain smooth

operation on Android devices. These ongoing maintenance activities ensure the long-term reliability, scalability, and availability of AgriGuard in real-world agricultural environments while minimizing downtime through planned update cycles.

Chapter 7: Conclusion and Future Work

AgriGuard is an AI-powered smart farming application designed to help farmers detect plant diseases quickly and accurately using image-based analysis. The application provides disease identification, treatment recommendations, weather updates, agricultural news, and an AI chatbot to support farmers in making informed decisions. With its simple and user-friendly interface, AgriGuard aims to reduce crop losses, improve productivity, and promote sustainable farming practices. Future enhancements such as support for more crops, offline functionality, IoT integration, smart irrigation, and multilingual support can further improve the system, making AgriGuard a comprehensive and reliable digital solution for modern agriculture.

7.1. Conclusion

AgriGuard is an intelligent plant disease detection and guidance system developed to support farmers and agricultural caretakers in identifying crop diseases quickly and accurately. The system combines image-based disease detection using deep learning with an AI-powered chatbot for agricultural guidance and support. By integrating machine learning, Android development, and API-based communication, AgriGuard provides a complete smart farming solution.The application helps users capture or upload plant images, detect possible diseases, check plant health status, and receive proper treatment suggestions. It also includes a caretaker chatbot feature that answers farming-related questions and provides useful recommendations for crop management. This improves decision-making, reduces crop loss, and increases agricultural productivity.

7.2. Future Work

Although AgriGuard provides an effective solution for plant disease detection and farmer assistance, several enhancements can be made in future versions. The application can be expanded to support a larger variety of crops and diseases with improved detection accuracy by training the model on more diverse datasets. Real-time pest detection and nutrient deficiency analysis can be integrated to provide comprehensive crop health

monitoring. Smart irrigation recommendations based on weather conditions and soil moisture data can help farmers optimize water usage. The system can also include multilingual support to make it accessible to farmers from different regions. An offline mode can be introduced so users can detect diseases without an internet connection. Furthermore, integration with IoT sensors, drone-based crop monitoring, and government agricultural services can transform AgriGuard into a complete smart farming platform that enhances productivity and promotes sustainable agriculture.

7.2.1 Real-Time Disease Detection

Future versions of AgriGuard can include real-time disease detection using the smartphone's camera. Instead of capturing and uploading an image manually, users will be able to point their camera at a plant leaf and receive instant disease identification with live analysis. The system will continuously process camera frames using advanced AI models to detect diseases in real time and highlight the affected areas on the screen. Along with disease detection, the application can immediately display the disease name, severity level, possible causes, and recommended treatment options. This enhancement will make the diagnosis process faster, more accurate, and more convenient, enabling farmers to identify plant health issues early and take timely preventive measures to reduce crop losses and improve agricultural productivity.

7.2.2 Multi-Language Support

To make AgriGuard more accessible and user-friendly, future versions can include multi-language support, allowing users to interact with the application in their preferred language. In addition to English, regional languages such as Urdu, Punjabi, Sindhi, Pashto, and Balochi can be supported to help farmers from different areas understand the application's features more easily. All menus, instructions, disease descriptions, treatment recommendations, weather updates, and AI chatbot responses can be translated into these languages. Voice assistance and text-to-speech features can also be integrated to assist farmers with limited literacy or those who prefer spoken guidance. This enhancement will improve usability, increase adoption among rural communities, and ensure that

language barriers do not prevent farmers from benefiting from modern agricultural technology.

7.2.3 Weather-Based Suggestions

Future versions of AgriGuard can integrate advanced weather forecasting APIs to provide intelligent, weather-based farming recommendations. By analyzing real-time and forecasted weather conditions such as temperature, humidity, rainfall, wind speed, and seasonal changes, the application can predict the likelihood of plant diseases and pest outbreaks before they occur. Based on these predictions, AgriGuard can send timely alerts and preventive recommendations, including the best time for irrigation, fertilizer application, pesticide spraying, and harvesting. The system can also warn farmers about extreme weather events such as heavy rainfall, heatwaves, frost, or storms that may affect crop health and yield. This proactive approach will help farmers take preventive measures in advance, minimize crop damage, improve resource management, and increase overall agricultural productivity.

7.2.4 Offline Functionality

Future versions of AgriGuard can include an offline mode to ensure that farmers in remote and rural areas with limited or no internet connectivity can continue using the application's essential features. A lightweight AI model can be stored directly on the user's device, allowing plant disease detection to be performed without requiring an active internet connection. Users will be able to capture or upload images of plant leaves and receive disease predictions along with basic treatment recommendations even when offline. The application can also locally store previously accessed disease information, farming guides, and user records for easy reference. Once an internet connection becomes available, the app can automatically synchronize stored data, update the disease detection model, refresh weather information, and download the latest agricultural news. This enhancement will improve the reliability, accessibility, and usability of AgriGuard, ensuring uninterrupted support for farmers regardless of network availability.

7.2.5 Expanded Crop Database

Future versions of AgriGuard can include an expanded crop database to support a wider variety of plants and plant diseases. By incorporating additional crop types such as cereals, vegetables, fruits, pulses, oilseed crops, and ornamental plants, the application will be able to serve a broader range of farmers and agricultural sectors. The disease database can also be enhanced by adding more disease categories, pest infestations, nutrient deficiencies, and environmental stress conditions. Regular updates to the database using verified agricultural research and expert recommendations will ensure that the system remains accurate and up to date. This enhancement will improve the application's disease detection capabilities, increase prediction accuracy across diverse crops, and make AgriGuard a more comprehensive and reliable solution for modern agriculture.

7.2.6 Government and Market Integration

Future versions of AgriGuard can integrate government agricultural services and market information to provide farmers with a more comprehensive farming support platform. The application can offer personalized fertilizer and pesticide recommendations based on the detected crop, disease type, soil conditions, and growth stage. It can also display real-time prices of fertilizers, pesticides, seeds, and other agricultural products from local markets, enabling farmers to compare costs and make informed purchasing decisions. In addition, AgriGuard can provide direct access to government agricultural support programs, subsidies, crop insurance schemes, training opportunities, and official farming guidelines. Farmers can receive notifications about new policies, financial assistance, and agricultural initiatives relevant to their region. This integration will help users access reliable information, reduce farming costs, improve decision-making, and strengthen the connection between farmers, agricultural markets, and government support services.

7.2.7 Voice Assistant Feature

Future versions of AgriGuard can include a **voice assistant feature** to make the application more accessible for farmers with limited literacy or those who prefer hands-

free interaction. The system can support voice commands, allowing users to navigate the application, capture plant images, ask farming-related questions, and request disease diagnosis simply by speaking. In addition, the application can provide voice responses by reading aloud disease names, treatment recommendations, weather updates, agricultural news, and AI chatbot replies in the user's preferred language. Support for regional languages such as Urdu, Punjabi, Sindhi, Pashto, and Balochi will further enhance usability for farmers across different regions. This feature will improve accessibility, reduce dependence on reading and typing, and enable farmers to interact with AgriGuard more naturally, efficiently, and confidently, regardless of their educational background.

These future improvements can make AgriGuard more powerful, practical, and beneficial for modern agriculture.

References

1. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016), Using Deep Learning for Image-Based Plant Disease Detection, Frontiers in Plant Science, Vol. 7, Article 1419, pp. 1–10. (Journal Paper)

2. Ferentinos, K. P. (2018), Deep Learning Models for Plant Disease Detection and Diagnosis, Computers and Electronics in Agriculture, Vol. 145, pp. 311–318. (Journal Paper)

3. Ian Goodfellow, Yoshua Bengio & Aaron Courville (2016), Deep Learning, MIT Press. Chapter 9, pp. 321–359. (Book Reference)

4. Android Developers, Android Studio User Guide, https://developer.android.com/studio, Last accessed: May 2026. (Website)

5. TensorFlow Team, TensorFlow Lite for Mobile and Embedded Devices, https://www.tensorflow.org/lite, Last accessed: May 2026. (Website)

6. OpenAI, ChatGPT API Documentation, https://platform.openai.com/docs, Last accessed: May 2026. (Website)

7. Groq Developers, Groq API Documentation for AI Integration, https://console.groq.com/docs, Last accessed: May 2026. (Website)

8. Jason Brownlee (2019), Deep Learning for Computer Vision, Machine Learning Mastery Publication. Chapter 5, pp. 145–198. (Book Reference)

9. Agriculture Research Service, Plant Disease Management Guidelines, https://www.ars.usda.gov, Last accessed: May 2026. (Website)

10. S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk & D. Stefanovic (2016), Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification, Computational Intelligence and Neuroscience, Vol. 2016, pp. 1–11. (Journal Paper)